

Рекурсия

Понятие рекурсивного алгоритма

- Рекурсия – это метод, сводящий общую задачу к некоторым задачам более узкого, простого типа.
- Хорошая реализация этого метода стала возможна с момента появления языков Алгол, Паскаль и других, которые поддерживают структуру программы, т.е. с тех пор, как появился язык локальных и глобальных переменных.

Сущность рекурсии

- Процедура или функция может содержать вызов других процедур или функций.
- В том числе процедура может вызвать саму себя.
- Никакого парадокса здесь нет — компьютер лишь последовательно выполняет встретившиеся ему в программе команды и, если встречается вызов процедуры, просто начинает выполнять эту процедуру.
- Не важно, какая процедура дала команду это делать.

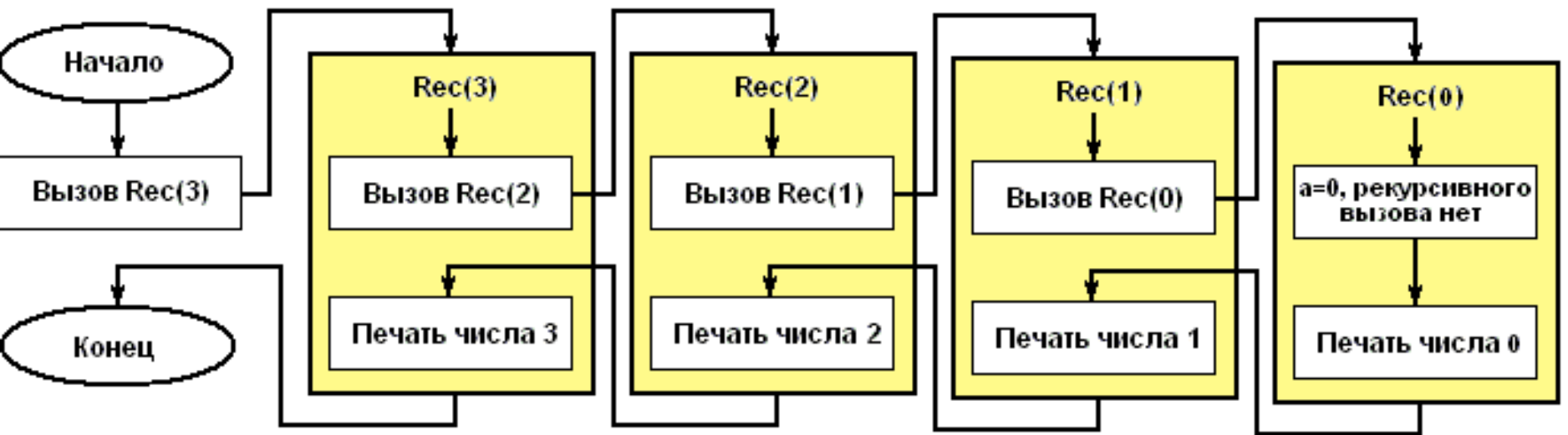
- Обычно в программировании под рекурсией понимают такую реализацию, в которой подпрограмма использует в своем теле вызов самой себя. Такие вызовы называют **рекурсивными**.
- Когда функция А в своем теле вызывает только одну рекурсивную функцию (саму себя), то говорят о **простой рекурсии**.
- Под **косвенной рекурсией** понимают явление, когда рекурсивные функции вызывают друг друга (например, функция А вызывает В, а функция В вызывает А).

- **Рекурсия** — метод определения класса объектов или методов предварительным заданием одного или нескольких (обычно простых) его базовых случаев или методов, а затем заданием на их основе правила построения определяемого класса или метода, ссылающегося прямо или косвенно на эти базовые случаи.

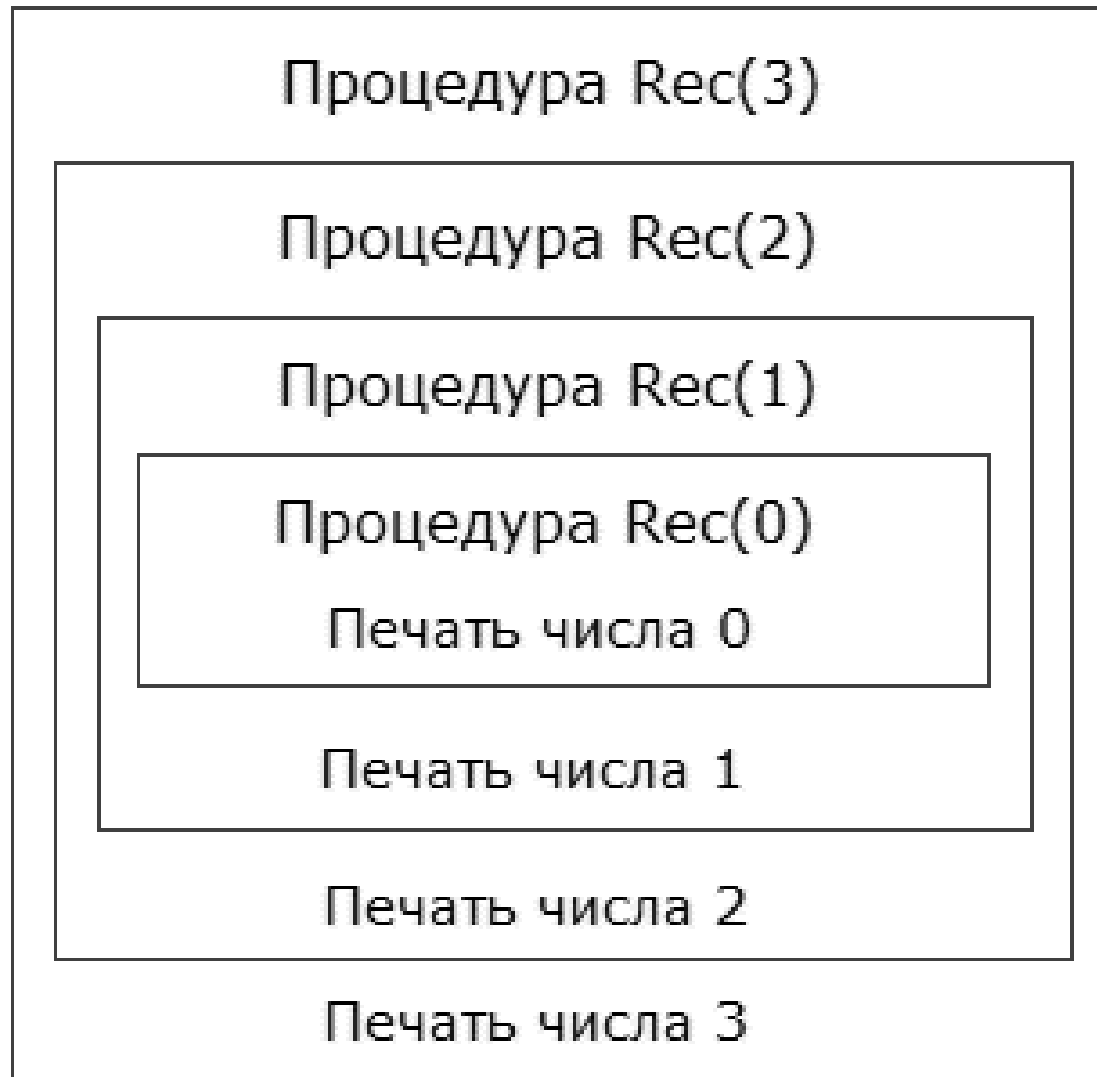
Пример рекурсивной процедуры

```
void Rec(int a)
{
    if (a>0) Rec(a-1);
    cout << a;
}
```

Блок схема работы рекурсивной процедуры $a=3$



Еще один визуальный образ работы рекурсивной процедуры $a=3$



Сложная рекурсия

- Возможна чуть более сложная схема:
функция А вызывает функцию В,
а та в свою очередь вызывает А.
Это называется **сложной рекурсией**.

Пример реализации сложной рекурсии

```
int A(int n); // прототипы функций  
int B(int n);
```

```
int A(int n) // определения функций  
{  
    if (n>0) return n + B(n-1)  
else return 0;  
}
```

```
int B(int n)  
{  
    if (n%2==0) return n + A(n-2)  
else return 1;  
}
```

1. Задачи с рекурсивной формулировкой

- Некоторые задачи являются рекурсивными по своему определению, поэтому их рекурсивные алгоритмы - это точные копии с соответствующего определения.
- Пример: Вычисление факториала натурального числа.

$$N! = \begin{cases} 1, & \text{при } N = 1 \\ N * (N-1)!, & \text{при } N > 1 \end{cases}$$

$$N! = \begin{cases} 1, & \text{при } N = 1 \\ N*(N-1)!, & \text{при } N > 1 \end{cases}$$

```
long factorial(int n)
{
if (n==1) return 1;
else return n*factorial(n-1);
}
```

2. Задачи, из постановки которых можно извлечь рекурсию

- Многие задачи не являются рекурсивными по определению, поэтому попытаемся свести некоторые из них к рекурсивным.
- Пример: Нахождение НОД (наибольшего общего делителя) двух натуральных чисел.

Нахождение НОД: алгоритм Евклида

- Пусть есть два целых числа a и b .
- Если $a = b$, то $\text{НОД}(a, b) = a$.
- Если $a > b$, то $\text{НОД}(a, b) = \text{НОД}(a - b, b)$.
- Если $a < b$, то $\text{НОД}(a, b) = \text{НОД}(a, b - a)$.

Пример $a=123$, $b=36$

a	b	Примечание.
123	36	так как $a > b$, то $a = a - b$
87	36	$a = a - b$
51	36	$a = a - b$
15	36	так как $b > a$, то $b = b - a$
15	21	$b = b - a$
15	6	$b = b - a$
9	6	$a = a - b$
3	6	$b = b - a$
3	3	так как $a = b$, то <u>НОД = a</u>

```
int NOD(int a, int b)
{
    if (a==b) {return a; }
    else if (a>b) {return NOD(a-b, b);}
    else {return NOD(a, b-a);}
}
```


3. Задачи, которые можно решить как частный случай обобщенной

- Суть заключается в том, что если нельзя извлечь рекурсию из постановки задачи, то можно расширить задачу, обобщить ее (например, введя дополнительный параметр).
- Удачное обобщение дает возможность увидеть рекурсию. После этого возвращаемся к частному случаю и решаем исходную задачу.
- **Пример.** Определить, является ли заданное натуральное число простым.

Пример. Определить, является ли заданное натуральное число простым

- Данную задачу можно обобщить, например, так: определить, верно ли, что заданное натуральное число N не делится ни на одно число, большее или равное M , но меньшее N . При этом значения числа M находятся в промежутке от 2 до N .
- Истина может быть в двух случаях:
- Если $M=N$;
- N не делится на M и истина для чисел $M+1$ и N .
- N не делится на M - это значит, что остаток от деления N на M не равен 0.

```
bool IsSimple(int M, int N)
{
if (M==N) return true;
else return (N%M != 0) && (IsSimple(M+1,N) );
}
```

Пример. Определить, является ли заданное натуральное число простым

- А теперь возвращаемся к исходной задаче:
- В качестве значения числа M надо взять 2.
- Таким образом, первое обращение к функции будет таким:
IsSimple(2, N), где N - это данное число.